

InfinityGAN: Training on Tiny Patches to Synthesize Infinite-Pixel Images

ICLR 2022 — a framework from UC Merced, Snap Inc., CMU, Yonsei, and Google Research for image synthesis at truly arbitrary size

Generative models have climbed steadily in resolution over the past few years, but there is a quiet ceiling that rarely gets named: almost every high-resolution GAN can only generate images at the exact size it was trained on. Ask a model like StyleGAN2 to output something larger than its training resolution and one of two things happens. Either compute and memory blow up quadratically with the side length until the run simply won't fit, or the global structure falls apart and the image dissolves into mechanical repetition. What the model has really learned is a single fixed-size picture, not a landscape that can extend forever.

InfinityGAN sets out to remove that ceiling directly. It is a framework for arbitrary-size, and in the limit infinite-pixel, image synthesis: the model trains only on small patches, yet at inference it can tile out an image of any size, patch by patch, while its memory footprint stays constant. Published at ICLR 2022 by researchers from UC Merced, Snap Inc., Carnegie Mellon, Yonsei University, and Google Research, its most striking claim is the hardware: both training and inference, at any output size, run on a single GTX TITAN X GPU.

It helps to see what "infinite" actually looks like first. The 512×4096 panorama below was assembled from 258 independently generated patches, with four different styles smoothly fused across its width — produced by a model that only ever saw 101×101 patches during training.



Figure 1. Spatial style fusion. This 512×4096 image is stitched from 258 independently generated patches, blending four styles left to right. Each patch is synthesized separately, yet they compose into one continuous, seam-free landscape — the core capability InfinityGAN is built around.

Paper: <https://arxiv.org/abs/2104.03963>

Code: <https://github.com/hubert0527/infinityGAN>

Project page: <https://hubert0527.github.io/infinityGAN/>

Why existing generators can't simply scale up

The trick behind InfinityGAN makes more sense once you notice something counterintuitive: generators like StyleGAN2 quietly rely on the image border to tell where they are. The zero-padding baked into their convolutions forms a fixed pattern at the edges during training, and the network memorizes that pattern as positional information.

The trouble comes when you ask for a larger image. The spatial size of the feature maps changes, the padding-derived position encoding shifts with it, and the center of a big image is now effectively infinitely far from any border — so the network receives no sensible positional signal there. The result is drastically repetitive texture and broken global structure. To synthesize truly infinite images you need positional information that stays valid infinitely far from any edge, together with a generator that can produce patches independently and still have them line up seamlessly.

Splitting an image into global appearance, structure, and texture

InfinityGAN's central move is to reframe image generation as disentangling three things: a global holistic appearance, local structural variation, and fine-grained texture. The generator G is built from two modules — a structure synthesizer G_S and a texture synthesizer G_T — that split the work of deciding what the scene looks like from the work of rendering its pixels.

The structure synthesizer G_S is a neural implicit function. It takes a global latent z_g that fixes the overall appearance of the whole image, a local latent z_l that carries local structural variation, and a grid of continuous coordinates c , and it outputs a structural representation z_S for exactly the region you asked for. Because it is driven by continuous coordinates, you can query any sub-region directly, even one that sits infinitely far from the origin in coordinate space. Coordinates are encoded with sine and cosine functions horizontally, exploiting the self-similarity of landscapes, and with a saturating tanh vertically, matching the transition from ground to sky.

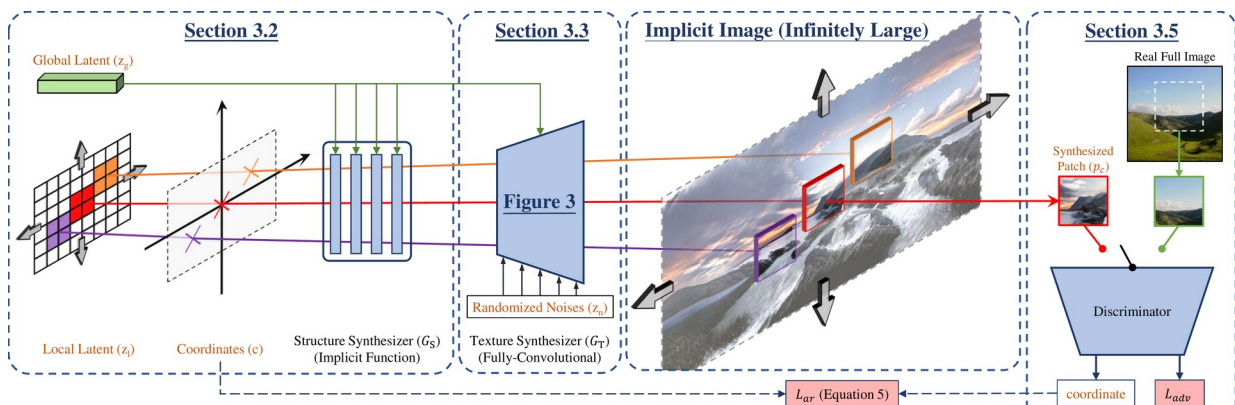


Figure 2. InfinityGAN overview. The structure synthesizer G_S (an implicit function) maps a global latent z_g , a local latent z_l , and continuous coordinates c to a structural representation; the fully-convolutional texture synthesizer G_T then renders a patch p_c from that representation plus randomized noise z_n . Because every pixel is determined by its coordinate alone, any region can be generated independently and tiled into an image of any size.

The texture synthesizer G_T is a fully-convolutional StyleGAN2 with all zero-padding removed. It takes the structural representation z_S , the global style z_g , and randomized noise z_n , and renders the final patch. The forward pass is just two steps: $z_S = G_S(z_g, z_l, c)$, then $p_c = G_T(z_S, z_g, z_n)$. Training adds R1 regularization, a path-length term, a mode-seeking diversity loss that discourages repetition, and an auxiliary loss that regresses the vertical coordinate, on top of the usual adversarial objective.

The key design: drop the padding so independent patches still line up

Since padding is the culprit, InfinityGAN removes all of it, producing a padding-free generator. Now the texture synthesizer learns texture and nothing else — it no longer smuggles in positional information — and all of the position comes from the coordinate-driven implicit function upstream.

The figure below contrasts the two designs. On the left, the conventional generator's padding makes the same location compute inconsistent pixel values across different patches, and that inconsistency spreads exponentially as the network deepens. On the right, the padding-free generator produces a consistent pixel value regardless of where the location falls in the receptive field. That property is exactly what lets independently generated patches be composed into one image with continuous feature values and no visible seams.

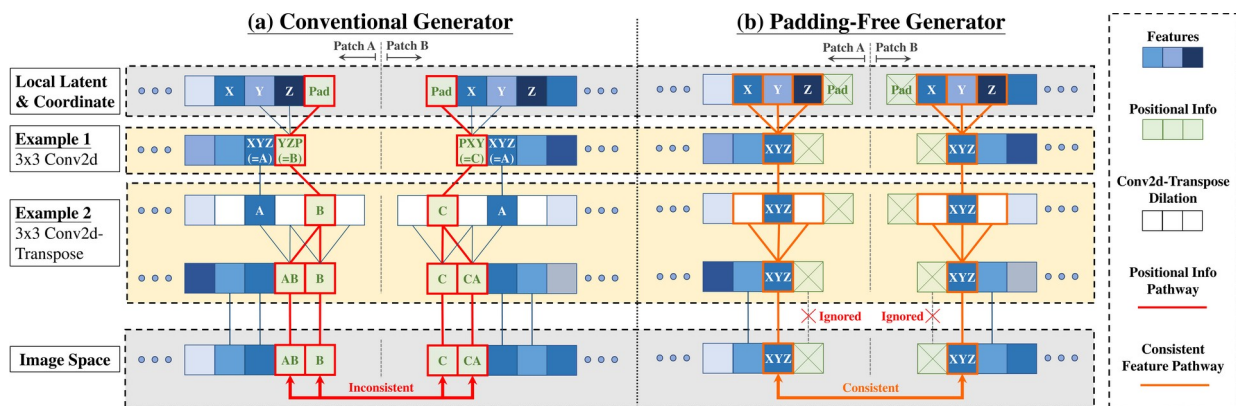


Figure 3. Padding-free generator. (Left) A conventional generator's zero-padding makes neighboring patches disagree on the pixel value at a shared location, and the inconsistent region grows exponentially with depth. (Right) The padding-free generator yields a consistent value at any position, enabling patches to be generated independently and then stitched seamlessly.

The bigger the image, the clearer the win: ScaleInv FID and a user study

How do you score a synthesized image several times larger than the training size, when no real images exist at that size to compare against? The authors propose ScaleInv FID: resize the large image back down to the training size with bilinear interpolation, then compute FID. $N \times$ ScaleInv FID means the metric is evaluated on images N times the training size. They compare InfinityGAN against a set of baselines on Flickr-Landscape, a dataset of 450,000 high-quality landscape images they collected for the purpose.

The trend is honest and telling. At $1 \times$ and $2 \times$, close to the training size, InfinityGAN does not lead — it was trained with a smaller field of view at a different resolution, a deliberately disadvantageous setting. But once the image is extended to $4 \times$ and beyond, its quality degrades the most slowly and it overtakes every baseline. More importantly, its memory is a constant $O(1)$ as the image grows, whereas StyleGAN2-style methods are $O(\text{size}^2)$ and run out of memory outright at the $16 \times$ setting. In a user study with 50 participants, more than 90% preferred InfinityGAN's results in every head-to-head comparison.

Table 1. ScaleInv FID on Flickr-Landscape (lower is better). InfinityGAN trails near the training size but pulls clearly ahead at 4× and 8×, at constant O(1) memory.

Method	1×	2×	4×	8×	Memory
SinGAN	4.21	57.10	145.12	210.22	O(size ²)
StyleGAN2+NCI	4.19	18.31	79.83	189.65	O(size ²)
InfinityGAN (ours)	21.84	28.83	61.41	121.18	O(1)

NCI: Non-Constant Input. At 8×, InfinityGAN scores 121.18 vs 189.65 for the strongest baseline; StyleGAN2 runs out of memory at the 16× setting.

A problem it solves along the way: arbitrary-size outpainting

Because the model can natively grow content in any direction, using it as a deep image prior for outpainting is a natural fit. The authors combine InfinityGAN with In&Out and test long-range outpainting on scenery subsets of Places365 and Flickr-Scenery. In the comparison below, Boundless and NS-outpaint start to break down as they move away from the given region, while In&Out + InfinityGAN keeps producing continuous, plausible content all the way out.

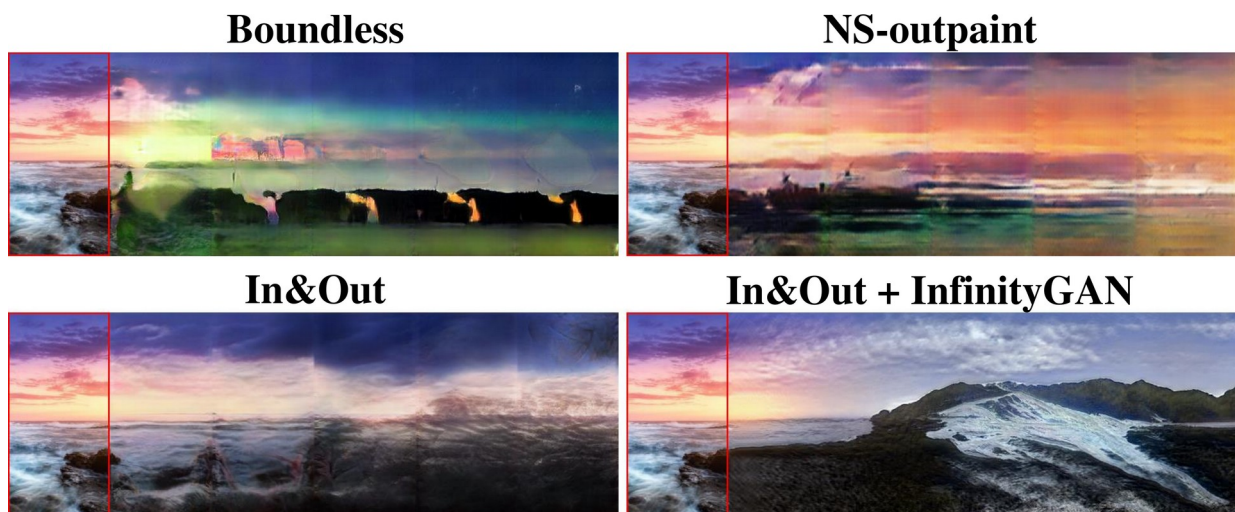


Figure 4. Long-range outpainting. The red box marks the given real region (256×128 pixels). Boundless and NS-outpaint degrade with distance, whereas In&Out + InfinityGAN keeps extending coherent, believable scenery.

The numbers back this up. In&Out + InfinityGAN drives outpainting FID down from In&Out's 23.57 / 30.34 to 9.11 / 15.31 on Places365 / Flickr-Scenery respectively (lower is better) — roughly halving the error and setting the state of the art on the task. And because patches are mutually independent and can be generated in parallel, at 8192×8192 pixels parallel batching cuts inference from 137.44 seconds to 19.09 seconds per image, a speed-up of up to 7.20×.

One framework, also good for panoramas and arbitrary-length inbetweening

Feed different local latents and styles into the same coordinates, or invert a real image at different positions, and the same framework naturally yields more: multi-modal outpainting, arbitrary-length cyclic panoramas, and arbitrary-length image inbetweening. The figure below shows inbetweening and cyclic-panorama results obtained by inverting real images; the top-row image is 256×2080 pixels.

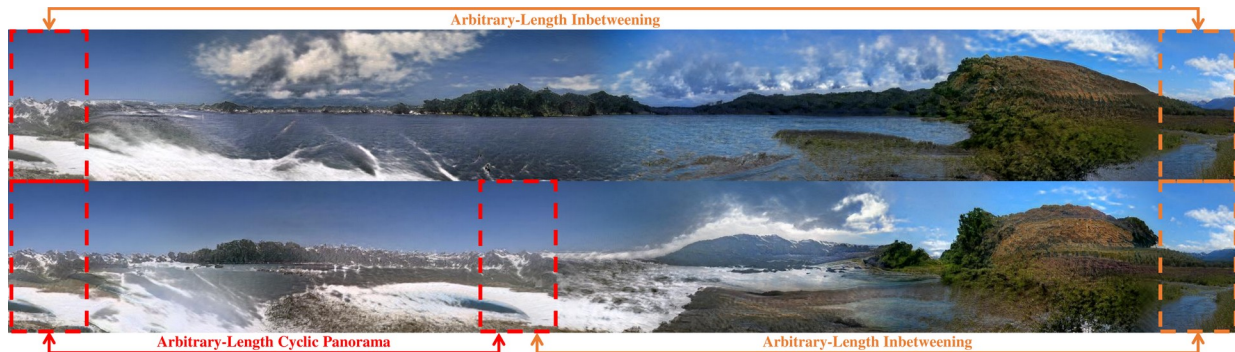


Figure 5. Image inbetweening with inverted latents. By inverting a real image at different positions, InfinityGAN synthesizes arbitrary-length cyclic panoramas and inbetweened images; because pixel values are guaranteed consistent, the composition is seamless. The top-row image is 256×2080 pixels.

Takeaway and honest boundaries

The core lesson of InfinityGAN is that infinite-pixel synthesis becomes tractable the moment you stop tying generation to a fixed output resolution. By separating the global appearance, the local structure via a coordinate-driven implicit function, and the texture via a padding-free convolutional generator, the model produces patches that are independent yet strictly consistent — so they tile into an image of any size at constant memory, and can be generated in parallel. The same design incidentally unlocks spatial style fusion, multi-modal outpainting, and arbitrary-length panoramas and inbetweening.

The authors are candid about the limits. ScaleInv FID captures a trend rather than a strictly comparable score, so the 1× numbers should not be read as a like-for-like loss. And on object-centric datasets with little vertical view variation, such as LSUN bridge and tower, the model often fills the top and bottom of the frame with blank padding texture. Still, as a framework that trains on small patches on a single consumer GPU yet synthesizes seamless, globally consistent images at arbitrary size, InfinityGAN offers a genuinely scalable path to making images bigger.